

System Mikrointerakcji: Architektura, Design i Implementacja Wzorców "Dymków" (Tooltips) oraz Popoverów w Środowisku Webowym

Streszczenie Wykonawcze

Współczesne interfejsy użytkownika (UI) balansują na cienkiej granicy między minimalizmem wizualnym a gęstością informacyjną. Aby rozwiązać ten konflikt, architekci systemów designu stosują technikę "progresywnego ujawniania" (ang. *progressive disclosure*), której kluczowymi komponentami są "Dymki" informacyjne (Tooltipy) oraz interaktywne kontenery pomocnicze (Popovery). Niniejszy raport stanowi wyczerpującą analizę techniczną, estetyczną i behawioralną systemu mikrointerakcji opartego na specyfikacji wizualnej "Dark Teal" (#002F2F, 90% opacyty) oraz rygorystycznych wymogach interakcji czasowej (opóźnienie 0.5s). Dokument ten dekonstruuje pozorne podobieństwo między tooltipem a popoverem, wykazując, że choć dzielą one warstwę wizualną, stanowią odrębne byty w kontekście modelu obiektowego DOM, dostępności (WCAG) oraz zarządzania stanem aplikacji. Raport adresuje również krytyczne wyzwanie adaptacji wzorców "hover" (najechnięcie) do środowisk dotykowych (mobile), proponując hybrydowy model "Toggletip", który zachowuje spójność estetyczną przy fundamentalnej zmianie paradygmatu wyzwalania akcji. Analiza obejmuje również zaawansowaną matematykę pozycjonowania elementów pływających (Floating UI), strategię zarządzania kontekstem stosu (Stacking Context) oraz psychofizykę percepcji ruchu w animacjach interfejsu.

1. Taksonomia i Filozofia Mikrointerakcji: Rozróżnienie Bytów

W profesjonalnej inżynierii frontendowej precyzja terminologiczna determinuje jakość implementacji. Choć użytkownik końcowy może postrzegać mały dymek z tekstem i większy dymek z przyciskiem jako ten sam element, z perspektywy architektury informacji są to dwa różne wzorce projektowe. Ignorowanie tej różnicy prowadzi do błędów dostępności i frustracji użytkownika.

1.1 Definicja i Rola Tooltipa (Dymek Informacyjny)

Tooltip, w swojej najczystszej formie, jest mechanizmem etykietowania. Jego jedynym celem jest odpowiedź na pytanie: "Czym jest ten element?" lub "Co się stanie, gdy go kliknę?". Jest to byt efemeryczny, ściśle powiązany z kursorem myszy lub fokusem klawiatury. Wymóg "krótki tekst (max 2 linie)" podany w specyfikacji jest nie tylko ograniczeniem stylistycznym, ale fundamentalną zasadą UX. Jeśli wyjaśnienie funkcji przycisku wymaga więcej niż dwóch linii tekstu, oznacza to zazwyczaj, że sama ikona lub etykieta przycisku jest źle

zaprojektowana, a tooltip próbuje "łatać" ten błąd. Tooltips nigdy nie powinny zawierać informacji krytycznych, niezbędnych do ukończenia zadania, ponieważ na urządzeniach dotykowych ich odkrywalność jest niska lub wymaga dodatkowej interakcji.

Z perspektywy modelu interakcji, tooltip jest "pasywny". Użytkownik nie wchodzi z nim w interakcję; użytkownik go *obserwuje*. Nie można zaznaczyć tekstu wewnątrz tooltipa (w większości implementacji), nie można w niego kliknąć (chyba że w celu podtrzymania widoczności), a jego zniknięcie jest automatyczne po opuszczeniu strefy aktywnej.

1.2 Definicja i Rola Popovera (Dymek Akcji/Wiedzy)

Popover, zdefiniowany w zapytaniu jako element mogący zawierać "przyciski, dłuższe teksty", to kontener interaktywny. W przeciwieństwie do tooltipa, Popover jest "aktywny". Może zawierać linki do dokumentacji, formularze (np. szybka edycja), czy też złożone dane (np. podgląd profilu użytkownika).

Kluczowa różnica leży w zarządzaniu uwagą (Focus Management). Gdy otwiera się tooltip, fokus klawiatury pozostaje na elemencie wyzwalającym (np. przycisku). Gdy otwiera się modalny Popover, fokus często musi zostać przeniesiony do wewnątrz dymka, aby umożliwić nawigację po przyciskach w nim zawartych. Implementacja Popovera jako Tooltipa (lub odwrotnie) jest błędem kategorii krytycznej w kontekście standardów W3C ARIA.

1.3 Tabela Porównawcza Wzorców

Poniższa tabela systematyzuje różnice wymagane do poprawnej implementacji obu bytów w ramach jednego systemu wizualnego "Dymków".

Cecha	Tooltip (Dymek Info)	Popover (Dymek Action)
Cel nadrzędny	Identyfikacja i Etykietowanie	Elaboracja i Interakcja
Wyzwalacz (Desktop)	mouseenter (Hover) + Opóźnienie	click (Kliknięcie)
Wyzwalacz (Mobile)	tap (Tapnięcie - Toggletip)	tap (Tapnięcie)
Zawartość	Tekst prosty (max 2 linie)	Rich Text, Przyciski, Linki
Interaktywność treści	Brak (Pasywna)	Pełna (Aktywna)
Zarządzanie Fokusem	Fokus pozostaje na Triggerze	Fokus może wejść do kontenera
Semantyka ARIA	role="tooltip"	role="dialog" lub role="region"
Powiązanie	aria-describedby	aria-details lub aria-labelledby
Zamykanie	mouseleave, Escape	click outside, przycisk "Zamknij", Escape

Analiza ta prowadzi do wniosku, że choć "look & feel" (wygląd i odczucie) obu elementów musi być identyczny dla zachowania spójności marki (Styl #002F2F), to "engine" (silnik) sterujący ich zachowaniem musi być warunkowy. System musi rozpoznawać intencję dewelopera: czy przekazano prosty ciąg znaków (uruchom logikę Tooltipa), czy komponent React/HTML (uruchom logikę Popovera).

2. Architektura Wizualna i Teoria Koloru

Specyfikacja narzuca rygorystyczny reżim wizualny: tło #002F2F przy 90% przezroczystości.

Jest to wybór estetyczny niosący za sobą konkretne implikacje dla czytelności i percepcji głębi.

2.1 Analiza Kolorymetryczna: Dark Teal

Kolor #002F2F to głęboki odcień morskiej zieleni (Dark Cyan/Teal).

- **Wartości RGB:** R:0, G:47, B:47.
- **Wartości HSL:** H:180, S:100%, L:9%.

Jest to kolor o bardzo niskiej luminancji (bliski czerni), ale z wyraźną, zimną temperaturą barwową. W psychologii koloru kojarzony jest z profesjonalizmem, spokojem i technologią.

Kontrast i Czytelność: Wymagany biały tekst (#FFFFFF) na tle #002F2F (zakładając 100% krycia) osiąga współczynnik kontrastu **15.57:1**.

- Norma WCAG AA (wymagane 4.5:1) – ZDALNA.
- Norma WCAG AAA (wymagane 7:1) – ZDALNA. Nawet przy małym rozmiarze czcionki (np. 12px, co jest standardem dla tooltipów), tekst będzie wybitnie czytelny. Jest to doskonały wybór pod kątem dostępności wizualnej.

2.2 Fizyka Przezroczystości (Opacity 90%)

Wprowadzenie wymogu 90% opacity (rgba(0, 47, 47, 0.9)) komplikuje sytuację w trybie "Dark Mode". Jeśli aplikacja, w której użyte są dymki, również posiada ciemne tło (np. #121212 lub ciemnoszary #1E1E1E z), dymek o kolorze #002F2F może zlać się z otoczeniem.

Problem zlewania się warstw: Przy 90% krycia, 10% tła przebija przez dymek. Na ciemnym tle, dymek traci swoje kontury. Użytkownik może mieć trudność z szybkim zidentyfikowaniem, gdzie kończy się etykieta, a zaczyna treść strony.

Rekomendacja Projektowa (Insight): Aby utrzymać wymóg 90% opacy, a jednocześnie zapewnić separację planów (Depth Separation), konieczne jest zastosowanie dwóch technik pomocniczych, które nie łamią specyfikacji, ale ją uzupełniają:

1. **Subtelny Obrys (Border):** Dodanie 1-pikselowego obrysu o jaśniejszym odcieniu teal lub półprzezroczystej bieli (np. rgba(255, 255, 255, 0.1)). Działa to jak "światło krawędziowe", definiując kształt dymka.
2. **Cień (Box Shadow):** Zastosowanie mocnego, rozmytego cienia (np. 0px 4px 16px rgba(0,0,0, 0.5)). Cień ten jest niezbędny, aby "unieść" dymek w osi Z (Z-axis) ponad warstwę interfejsu. W przypadku Popoverów, cień powinien być jeszcze głębszy, sugerując wyższą elewację.
3. **Backdrop Blur (Rozmycie Tła):** Zastosowanie CSS backdrop-filter: blur(4px). To nowoczesna technika (znana z iOS i Windows Acrylic), która sprawia, że treść pod dymkiem staje się nieczytelna (rozmyta). Zwiększa to czytelność białego tekstu na dymku, ponieważ eliminuje "szum" wizualny przebijający z tła, jednocześnie zachowując efekt półprzezroczystości.

2.3 Geometria Zaokrągleń i Strzałki

Wymóg "zaokrąglone rogi" i "mała strzałka" sugeruje estetykę organiczną, przyjazną użytkownikowi.

Promień Zaokrąglenia (Border Radius): Dla elementów typu Tooltip (mała wysokość), promień powinien być skorelowany z wysokością linii tekstu.

- Jeśli dymek ma padding 8px i tekst 14px (łącznie ok. 30-34px wysokości), promień 4px do 6px jest standardem branżowym (Bootstrap, Material).

- Alternatywą jest styl "Pill" (kapsułka), gdzie border-radius: 999px. Jest to jednak ryzykowne przy tekstach dwuliniowych (wygląda nienaturalnie), dlatego rekomendujemy border-radius: 6px jako złoty środek między nowoczesnością a funkcjonalnością dla bloków tekstu.

Konstrukcja Strzałki (Arrow): Tradycyjna metoda CSS (trójkąt z border-color) jest niewystarczająca przy 90% opacności i zaokrąglonych rogach, ponieważ trudno na niej odwzorować ten sam stopień przezroczystości i cienia co na głównym kontenerze bez artefaktów wizualnych.

- **Rozwiązanie SVG:** Najlepszą metodą jest użycie małego elementu SVG jako strzałki. Pozwala to na idealne dopasowanie koloru #002F2F oraz, co kluczowe, na lekkie zaokrąglenie połączenia strzałki z dymkiem (tzw. "smooth union"), co podnosi postrzeganą jakość interfejsu (premium feel).

3. Dynamika Interakcji Czasowej: Opóźnienie 0.5s

Wymóg "Opóźnienie: 0.5s" jest precyzyjny i ma głębokie uzasadnienie w psychofizyce interfejsów, znane jako "Hover Intent" (Intencja Najejchania).

3.1 Mechanika Intencji

Użytkownik często przesuwając kursor przez ekran, aby dotrzeć do celu A, mijając po drodze elementy B, C i D. Gdyby tooltipy pojawiały się natychmiast (delay: 0s), przelot kursora nad paskiem narzędzi spowodowałby stroboskopowy efekt migania dymków (efekt "choinki"). Jest to męczące poznawczo i odwraca uwagę.

Opóźnienie 500ms (0.5s) działa jak filtr dolnoprzepustowy. System zakłada, że jeśli kursor zatrzymał się (lub porusza się bardzo wolno) w obrębie elementu przez pół sekundy, użytkownik celowo szuka informacji.

3.2 Implementacja Logiki Debouncingu (Odbicia)

Technicznie, implementacja tego opóźnienia nie może polegać na prostej animacji CSS transition-delay, ponieważ:

1. Tooltip nadal renderowałby się w DOM (nawet jeśli niewidoczny), co może wpływać na wydajność.
2. Czytniki ekranowe mogłyby odczytać treść tooltipa, zanim użytkownik faktycznie "zdecyduje" się go zobaczyć.

Prawidłowy model w JavaScript (React/Vue/Vanilla) wygląda następująco:

1. Zdarzenie mouseenter: Uruchom Timer (setTimeout) na 500ms.
2. Zdarzenie mouseleave: Wyczyść Timer (clearTimeout).
3. Jeśli Timer dobiegnie końca: Zmień stan na isOpen = true (zamontuj komponent).
4. Zdarzenie mouseleave (gdy otwarty): Zmień stan na isOpen = false (ewentualnie z krótkim opóźnieniem "grace period" na wyjście).

Krytyczny Wyjątek - "Warmup" (Rozgrzewka): W zaawansowanych systemach (jak Material Design), jeśli użytkownik otworzył już jeden tooltip, a następnie szybko przesuwając kursor na sąsiedni element z tooltipem, opóźnienie 0.5s powinno zostać **anulowane**. Użytkownik wszedł w tryb "eksploracji". Pierwszy dymek wymaga 500ms, ale kolejne powinny pojawiać się natychmiast (lub np. po 100ms), aby zapewnić płynność przeglądania. Implementacja tego

wymaga globalnego stanu "Tooltip Group".

4. Adaptacja Mobilna: Paradygmat "Tapnięcia"

Wymóg "Mobile: tapnięcie" stanowi największe wyzwanie UX. Na desktopie hover jest stanem "darmowym" – nie wymaga kliknięcia, nie uruchamia akcji. Na mobile, tap jest ekwiwalentem click – zazwyczaj uruchamia akcję.

4.1 Konflikt Akcji

Jeśli przycisk "Usuń" ma tooltip "Usuwa trwale rekord", to na desktopie użytkownik najechał (widzi ostrzeżenie) i klika. Na mobile, jeśli użytkownik tapnie przycisk, aby zobaczyć tooltip, może niechcący usunąć rekord. Jest to konflikt między "podglądem" a "wykonaniem".

Snippet sugeruje, że długie przytrzymanie (Long Press) jest problematyczne na iOS (konflikt z menu systemowym). Zatem "Tapnięcie" jest jedyną opcją, ale musi być wdrożone inteligentnie: **Strategia Toggletip**: Dla elementów, które mają akcję krytyczną, tooltip na mobile nie powinien być podpięty pod sam element, lecz pod ikonę pomocniczą (np. małe (i) lub ? obok etykiety).

Tapnięcie w ikonę wyzwala dymek (Toggle: włącz/wyłącz).

Dla elementów, których *jedyną* funkcją jest informacja (np. ikona statusu), tapnięcie działa jako przełącznik widoczności dymka.

4.2 Logika Zamykania na Mobile

Na desktopie dymek znika po zjechaniu kursorem (mouseleave). Na mobile kursor nie "zjeżdża". Dymek po otwarciu (tapnięciu) pozostaje "lepki" (sticky). Wymaga to implementacji nasłuchu **"Click Outside"**.

1. Użytkownik tapie element -> Dymek się otwiera.
2. Użytkownik tapie *gdziekolwiek indziej* na ekranie -> Dymek się zamyka.
3. Użytkownik scrolluje stronę -> Dymek powinien się zamknąć lub (w przypadku Floating UI) przesunąć się wraz z elementem, choć bezpieczniej jest go zamknąć, by nie zasłaniał widoku.

5. Inżynieria Pozycjonowania: Algorytmy Floating UI

Wymóg "Pozycjonowanie: nad/pod/obok" brzmi trywialnie, ale matematyka stojąca za utrzymaniem elementu w widoku (viewport) jest złożona. Statyczne przypisanie CSS (np. bottom: 100%) zawiedzie, gdy element znajdzie się przy krawędzi ekranu.

5.1 Silnik Pozycjonujący

Rekomendujemy użycie biblioteki algorytmicznej (standard branżowy: **Floating UI**, dawniej Popper.js). Rozwiązuje ona problemy, których ręczne "if-owanie" w JavaScript jest podatne na błędy.

Kluczowe Middleware (Pośredniki logiczne):

1. **Offset (Przesunięcie)**: Dymek nie może dotykać elementu. Potrzebny jest odstęp (np. 8px) na strzałkę.
2. **Flip (Odwracanie)**: Jeśli preferowana pozycja to "Nad" (Top), ale brakuje miejsca u góry

ekranu (np. przycisk jest przy samej belce nawigacyjnej), algorytm automatycznie zmienia pozycję na "Pod" (Bottom).

3. **Shift (Przesuwanie):** Jeśli dymek jest szerszy niż element i wystaje poza prawą krawędź ekranu (na mobile to częste), algorytm przesuw go w lewo, wzdłuż osi X, aby w całości zmieścił się w oknie.
4. **Arrow (Strzałka):** Gdy dymek zostanie przesunięty (Shift), strzałka musi zostać "odczepiona" od środka dymka i przesunięta tak, by nadal wskazywała na środek elementu wyzwającego. To dynamiczne obliczanie pozycji strzałki jest kluczowe dla zachowania spójności wizualnej.

5.2 Strategia "React Portal" i Z-Index

Dymki są podatne na tzw. "pułapkę Z-Index" (Stacking Context Trap). Jeśli przycisk, na którym jest tooltip, znajduje się w kontenerze z overflow: hidden (np. przewijana tabela) lub ma niskie z-index, tooltip zostanie obcięty lub schowany pod innymi elementami interfejsu.

Rozwiązanie Architektoniczne: Dymek nie powinien być renderowany w drzewie DOM jako dziecko przycisku. Należy użyć techniki **Portal** (w React createPortal), aby fizycznie wyrenderować kod HTML dymka na samym końcu znacznika <body>. Dzięki temu:

1. Dymek ucieka z wszelkich ograniczeń overflow: hidden rodziców.
2. Możemy nadać mu globalny, bardzo wysoki z-index (np. 1500), gwarantując, że będzie "pływał" nad wszystkimi modalami, nagłówkami i treścią. Pozycjonowanie odbywa się wtedy absolutnie względem okna przeglądarki, a nie względem rodzica, co wymaga synchronizacji koordynatów w każdym cyklu renderowania (co zapewnia Floating UI autoUpdate).

6. Animacja i "Premium Feel"

Aby dymki nie pojawiały się w sposób "sztywny", należy zastosować subtelną animację wejścia. Badania oraz wytyczne Apple HIG sugerują użycie krzywych Beziera zamiast animacji liniowych.

6.1 Krzywa Beziera

Zamiast standardowego ease-out, proponujemy niestandardową krzywą sześcienną, która daje efekt lekkiego "wysokoku" i miękkiego lądowania.

- **Krzywa:** cubic-bezier(0.16, 1, 0.3, 1) (zbliżona do "Ease Out Expo").
- **Czas:** Animacja powinna być szybka. Opóźnienie wyzwolenia to 0.5s, ale sama animacja pojawiania się powinna trwać ok. **200ms**. Dłuższa animacja sprawi wrażenie "mulenia" interfejsu.

6.2 Choreografia CSS

Rekomendujemy połączenie zmiany przezroczystości (Opacity) z minimalnym przesunięciem (Transform) lub skalowaniem (Scale).

- **Start:** opacity: 0, transform: scale(0.95) translateY(4px).
- **Koniec:** opacity: 1, transform: scale(1) translateY(0). Daje to efekt, jakby dymek "wyrastał" z elementu w stronę użytkownika. Użycie will-change: transform, opacity jest

zalecane dla optymalizacji wydajności renderingu na słabszych urządzeniach mobilnych.

7. Dostępność (Accessibility/A11y) - Wymogi Prawne i Etyczne

Implementacja ARIA Label, o którą prosi użytkownik, to wierzchołek góry lodowej.

7.1 Semantyka ARIA

- **Tooltip:**
 - Trigger: Musi posiadać atrybut `aria-describedby="ID_DYMKA"`. To powoduje, że czytnik ekranu po najeźdzeniu na przycisk przeczyta: "Przycisk Zapisz. Podpowiedź: Zapisuje zmiany na dysku".
 - Kontener Dymka: Musi posiadać `id="ID_DYMKA"` oraz `role="tooltip"`.
 - **Ważne:** Jeśli tooltip zawiera dokładnie ten sam tekst co przycisk, nie należy go używać w ARIA, aby uniknąć redundancji ("Zapisz. Podpowiedź: Zapisz").
- **Popover:**
 - Trigger: Może używać `aria-expanded="true/false"` oraz `aria-controls="ID_POPOVERA"`.
 - Kontener: `role="dialog"` (jeśli modalny) lub `role="region"`. Wymaga `aria-labelledby` wskazującego na nagłówek popovera.

7.2 Obsługa Klawiatury (WCAG 2.1)

Użytkownicy nawigujący wyłącznie klawiaturą (Tab) muszą mieć dostęp do tooltipów.

- Tooltip musi pojawić się po otrzymaniu fokusa (onFocus).
- **Klawisz Escape:** Użytkownik musi mieć możliwość zamknięcia tooltipa klawiszem Esc bez przesuwania fokusa. Jest to wymóg kryterium sukcesu WCAG 1.4.13 (Content on Hover or Focus).

7.3 Bezpieczny Trójkąt (Safe Triangle)

Jest to problem użyteczności. Jeśli użytkownik najeżdża na tooltip, aby skopiować z niego tekst (w przypadku Popovera) lub po prostu lepiej go zobaczyć (lupy ekranowe), dymek nie może zniknąć w momencie, gdy kursor opuści przycisk i wejdzie na obszar dymka. Technicznie rozwiązuje się to poprzez:

1. Dodanie niewidzialnego "mostu" (paddingu) między elementem a dymkiem.
2. Lub logikę JavaScript, która sprawdza, czy kursor znajduje się *albo* na triggerze *albo* na dymku przed zamknięciem.

8. Specyfikacja Implementacyjna (Kod i Struktura)

Poniżej przedstawiono syntezę wymagań w formie specyfikacji dla zespołu deweloperskiego.

Tabela 4: Zmienne Systemowe (Design Tokens)

Token (Zmienna)	Wartość	Opis Funkcjonalny
--dymek-bg	rgba(0, 47, 47, 0.9)	Tło "Dark Teal" z przezroczystością. Wymaga backdrop-filter.
--dymek-text	#FFFFFF	Kolor tekstu. Wymagany wysoki kontrast.
--dymek-radius	6px	Promień zaokrąglenia rogów.
--dymek-arrow	8px	Szerokość podstawy strzałki SVG.
--dymek-z-index	9999	Warstwa najwyższa (powyżej modali).
--anim-ease	cubic-bezier(0.16, 1, 0.3, 1)	Krzywa animacji "Premium".
--anim-duration	200ms	Czas trwania wejścia/wyjścia.
--delay-open	500ms	Opóźnienie intencji (tylko desktop).

Schemat Logiczny Komponentu (Pseudo-kod)

```
/**
 * Główny hook zarządzający logiką dymka
 */
function useDymekLogic({ mode = 'tooltip', delay = 500 }) {
  const [isOpen, setIsOpen] = useState(false);
  const timerRef = useRef(null);

  // Obsługa wejścia myszy (Desktop)
  const handleMouseEnter = () => {
    if (mode === 'popover') return; // Popovery otwieramy tylko
    klikaniem
    timerRef.current = setTimeout(() => {
      setIsOpen(true);
    }, delay);
  };

  // Obsługa wyjścia myszy
  const handleMouseLeave = () => {
    if (mode === 'popover') return; // Popovery zamykamy klikaniem na
    zewnątrz
    clearTimeout(timerRef.current);
    setIsOpen(false);
  };

  // Obsługa kliknięcia (Mobile / Popover)
  const handleClick = (e) => {
    // Toggle tip logic
    setIsOpen(prev => !prev);
  };
}
```



```
// Obsługa klawiatury (Ally)
const handleFocus = () => setIsOpen(true);
const handleBlur = () => setIsOpen(false);
const handleEscape = () => setIsOpen(false);

return { isOpen, handlers: {...} };
}
```

9. Wnioski i Rekomendacje Końcowe

Projekt systemu "Tooltipy & Popovery" opartego na kolorze #002F2F i opóźnieniu 0.5s wymaga wyjścia poza prostą stylowanie CSS. Aby spełnić wymogi współczesnego web developmentu, system musi być **inteligentny**:

1. **Musi rozróżniać kontekst:** Desktop (Hover Intent) vs Mobile (Explicit Tap).
2. **Musi być świadomy przestrzeni:** Automatyczne odwracanie (Flip) i przesuwanie (Shift) przy krawędziach ekranu.
3. **Musi być dostępny:** Ścisła integracja z drzewem dostępności przeglądarki (ARIA) i obsługa klawiatury.
4. **Musi być estetyczny:** Zastosowanie "blur" i cieniowania, aby skompensować ryzyko niskiego kontrastu krawędzi przy 90% przezroczystości na ciemnych tłach.

Wdrożenie powyższych zaleceń pozwoli stworzyć komponent, który nie tylko wygląda profesjonalnie ("premium feel"), ale jest również odporny na błędy użytkownika i skalowalny w złożonych aplikacjach webowych.

Cytowane prace

1. Tooltip vs. Popover - touch support · Issue #3530 · mui/base-ui - GitHub, <https://github.com/mui/base-ui/issues/3530>
2. ARIA: tooltip role - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Roles/tooltip_role
3. Creating an accessible tooltip - Designing accessible components using the ustwo Inclusivity Principles, <https://engineering.ustwo.com/articles/creating-an-accessible-tooltip/>
4. Modal vs Popover vs Drawer vs Tooltip: When to Use Each (2025 Guide) - UX Patterns, <https://uxpatterns.dev/pattern-guide/modal-vs-popover-guide>
5. Guidelines | Popover | Red Hat design system, <https://ux.redhat.com/elements/popover/guidelines/>
6. Dark mode UI: 9 design considerations you can't ignore - Five Jars, <https://fivejars.com/insights/dark-mode-ui-9-design-considerations-you-cant-ignore/>
7. Dark Mode Design: A Practical Guide With Tips and Examples - UX Design Institute, <https://www.uxdesigninstitute.com/blog/dark-mode-design-practical-guide/>
8. Dark mode UI design: Best practices and examples - LogRocket Blog, <https://blog.logrocket.com/ux-design/dark-mode-ui-design-best-practices-and-examples/>
9. How to design a Tooltip: Anatomy, best practices & UX tips - Setproduct, <https://www.setproduct.com/blog/tooltip-ui-design>
10. useClickOutside() - Webrix.js - Amdocs, <https://webrix.amdocs.com/docs/hooks/useclickoutside>
11. useClickAway React Hook - useHooks, <https://usehooks.com/useclickaway>
12. computePosition - Floating UI, <https://floating-ui.com/docs/compute-position>
13. Getting started with Floating UI - LogRocket Blog, <https://blog.logrocket.com/getting-started-floating-ui/>
14. flip - Floating UI,

<https://floating-ui.com/docs/flip> 15. size | Floating UI, <https://floating-ui.com/docs/size> 16. The Z-Index Problem. Why Stacking Contexts Are a CSS... | by Shubham Sharma | Medium, <https://medium.com/@ss-tech/the-z-index-problem-91226fb74955> 17. cubic-bezier() - CSS - MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/easing-function/cubic-bezier> 18. How to read cubic-bezier curves - Val Head, <https://valhead.com/2017/02/13/how-to-read-cubic-bezier-curves/>